

UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL MENDOZA

CARRERA: TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

NOTIFICACIÓN AUTOMATIZADA DE PARCHES DE VULNERABILIDADES: MONITOREO, PRIORIZACIÓN Y ALERTAS TEMPRANAS

Trabajo final de carrera para optar por el título de Técnico Universitario en
Programación

Autores:

Barros Hernandez, Maria Emilia

Frankenberger, Federico Agustin

Maricchiolo, Guadalupe

Mendoza, Argentina

2026

RESUMEN

La gestión de vulnerabilidades en entornos tecnológicos actuales presenta desafíos asociados al volumen creciente de información disponible y a la demora en los procesos manuales de análisis y respuesta. En este contexto, el presente trabajo propone el desarrollo de GEF Secure, un sistema automatizado orientado a la detección, priorización y notificación de vulnerabilidades de seguridad.

La solución se implementa mediante un pipeline automatizado desarrollado en la plataforma n8n, ejecutado sobre una arquitectura basada en contenedores Docker. El sistema integra información proveniente de fuentes externas, como GitHub Advisory y el catálogo CISA Known Exploited Vulnerabilities (KEV), y la correlaciona con activos internos almacenados en una base de datos PostgreSQL.

A partir de estos datos, se aplica un conjunto de reglas de procesamiento implementadas en JavaScript que permiten diferenciar entre el riesgo técnico, evaluado mediante el sistema CVSS, y el riesgo operativo asociado a la explotación activa de vulnerabilidades.

Los resultados obtenidos evidencian una mejora significativa en los tiempos de respuesta ante la detección de vulnerabilidades, reflejada en la reducción del MTTR (Mean Time To Respond), así como una mayor trazabilidad en la gestión de eventos de seguridad.

En conclusión, el sistema desarrollado permite automatizar el proceso de monitoreo de vulnerabilidades, contribuyendo a mejorar la eficiencia en la gestión de riesgos y facilitando la adopción de enfoques más proactivos en ciberseguridad.

ÍNDICE

RESUMEN.....	2
ÍNDICE.....	3
INTRODUCCIÓN.....	5
1 . CONTEXTO.....	5
2. PLANTEAMIENTO DEL PROBLEMA.....	5
3. JUSTIFICACIÓN.....	6
4. OBJETIVOS.....	7
4.1. OBJETIVO GENERAL.....	7
4.2. OBJETIVOS ESPECÍFICOS.....	7
5. ALCANCES Y LIMITACIONES.....	8
5.1. ALCANCES.....	8
5.2. LIMITACIONES.....	8
MARCO TEÓRICO.....	10
1. SEGURIDAD INFORMÁTICA.....	10
2 AMENAZAS Y VULNERABILIDADES.....	10
3. MODELOS DE SEGURIDAD.....	11
4. HERRAMIENTAS DE ANÁLISIS.....	12
MARCO METODOLÓGICO.....	14
1. TIPO DE INVESTIGACIÓN.....	14
2. DISEÑO DE INVESTIGACIÓN.....	15
3. TÉCNICAS DE RECOLECCIÓN DE DATOS.....	15
4. INSTRUMENTOS.....	16
DESARROLLO DE ESTUDIO.....	18
1. ARQUITECTURA DEL SISTEMA.....	18
1.1 ARQUITECTURA GENERAL.....	18
1.2 MODELO DE DATOS.....	18
2 HERRAMIENTAS UTILIZADAS.....	19
2.1 AUTOMATIZACIÓN DE WORKFLOWS.....	19
2.2 BASE DE DATOS.....	20
2.3 BACKEND.....	20
2.4 FRONTEND.....	21
2.5 INFRAESTRUCTURA.....	21
2.6 NOTIFICACIONES.....	21
RESULTADOS.....	23
1. FUNCIONALIDADES IMPLEMENTADAS.....	23
2. EVALUACIÓN DEL SISTEMA.....	24
3. MÉTRICAS OBTENIDAS.....	24
4. VALIDACIÓN DE OBJETIVOS.....	25
DISCUSIÓN.....	26
CONCLUSIONES.....	28
RECOMENDACIONES.....	30

REFERENCIAS BIBLIOGRÁFICAS.....	32
ANEXOS.....	33
Anexo A — Cronograma de implementación.....	33
Anexo B — Esquema de base de datos.....	36
Anexo C — Workflow de automatización en n8n.....	46
Anexo D — Ejemplo de notificación en Slack.....	53
Anexo E — Endpoints del backend.....	55
Anexo F — Ejemplo de datos procesados.....	55
Anexo G — Interfaz de usuario (Frontend).....	58

INTRODUCCIÓN

1 . CONTEXTO

En los últimos años, el incremento sostenido en la cantidad de vulnerabilidades de seguridad reportadas ha generado nuevos desafíos para la gestión de riesgos en entornos tecnológicos. Organizaciones de distintos sectores enfrentan diariamente la necesidad de analizar grandes volúmenes de información provenientes de múltiples fuentes, con el objetivo de identificar aquellas vulnerabilidades que representan un riesgo real para sus activos.

2. PLANTEAMIENTO DEL PROBLEMA

En la actualidad, la gestión de vulnerabilidades de seguridad se ha convertido en un proceso cada vez más complejo debido al crecimiento exponencial en la cantidad de vulnerabilidades reportadas a nivel global. Organizaciones de distintos sectores deben analizar continuamente información proveniente de múltiples fuentes, como bases de datos públicas, reportes de seguridad y catálogos de vulnerabilidades explotadas activamente.

Este aumento en el volumen de información genera un fenómeno de sobrecarga que dificulta la identificación de aquellas vulnerabilidades que representan un riesgo real para los activos de una organización. En muchos casos, los equipos de seguridad deben revisar manualmente grandes cantidades de registros, lo que incrementa la probabilidad de errores y retrasa la toma de decisiones.

A su vez, los procesos tradicionales de gestión de vulnerabilidades suelen depender de tareas manuales para la recopilación, análisis y priorización de información. Esta dependencia introduce demoras significativas en los tiempos de respuesta, especialmente frente a vulnerabilidades críticas que requieren atención inmediata. Como consecuencia, se incrementa el riesgo de explotación y se reduce la capacidad de respuesta ante incidentes de seguridad.

Otro aspecto relevante es la dificultad para correlacionar la información externa sobre vulnerabilidades con el contexto interno de una organización, particularmente en relación con los activos afectados. Sin un mecanismo automatizado que permita realizar esta correlación, resulta complejo determinar el impacto real de una vulnerabilidad en un entorno específico.

En este contexto, surge la necesidad de desarrollar soluciones que permitan automatizar el proceso de monitoreo, análisis y priorización de vulnerabilidades, integrando múltiples fuentes de información y reduciendo la dependencia de intervenciones manuales. La ausencia de este tipo de soluciones limita la eficiencia de los equipos de seguridad y dificulta la adopción de enfoques proactivos en la gestión de riesgos.

3. JUSTIFICACIÓN

La creciente complejidad en la gestión de vulnerabilidades de seguridad, producto del aumento constante en la cantidad de amenazas y fuentes de información disponibles, evidencia la necesidad de desarrollar soluciones que permitan optimizar los procesos de análisis y respuesta. En este contexto, la automatización se presenta como una estrategia fundamental para mejorar la eficiencia operativa en la gestión de la seguridad informática.

El desarrollo de un sistema automatizado como el propuesto en este trabajo permite reducir la dependencia de procesos manuales, disminuyendo los tiempos de análisis y respuesta frente a vulnerabilidades críticas. Esto contribuye directamente a la reducción del riesgo de explotación y mejora la capacidad de las organizaciones para gestionar incidentes de seguridad de manera oportuna.

Desde el punto de vista técnico, la integración de múltiples fuentes de información, como bases de datos de vulnerabilidades y catálogos de explotación activa, permite enriquecer el análisis y mejorar la priorización de riesgos. Asimismo, la correlación de estos datos con el contexto interno de una organización posibilita una evaluación más precisa del impacto potencial de cada vulnerabilidad.

Por otra parte, el sistema propuesto incorpora mecanismos de trazabilidad y registro de información que facilitan el seguimiento de las acciones realizadas, lo cual resulta relevante tanto para la gestión operativa como para el cumplimiento de requisitos de auditoría y normativas de seguridad.

Desde una perspectiva académica, este trabajo aporta una implementación práctica de conceptos vinculados a la automatización, la gestión de vulnerabilidades y la orquestación de procesos, integrando herramientas y tecnologías ampliamente utilizadas en el ámbito profesional. Esto permite no solo validar la viabilidad técnica de la solución, sino también generar un caso de estudio aplicable en contextos educativos.

En conjunto, la propuesta contribuye a mejorar la eficiencia en la gestión de vulnerabilidades, promoviendo un enfoque más sistemático, automatizado y orientado a la reducción de riesgos en entornos tecnológicos.

4. OBJETIVOS

4.1. OBJETIVO GENERAL

Desarrollar un sistema integral para la gestión automatizada de vulnerabilidades de seguridad, que permita la detección, análisis, priorización y notificación de amenazas, integrando múltiples fuentes de información y reduciendo la dependencia de procesos manuales, mediante una arquitectura basada en automatización, almacenamiento estructurado y capas de aplicación.

4.2. OBJETIVOS ESPECÍFICOS

- Diseñar un pipeline automatizado que permita la recolección de información sobre vulnerabilidades desde fuentes externas. (Véase Anexo
- Integrar datos provenientes de distintas bases de vulnerabilidades y correlacionarlos con activos internos de una organización.
- Implementar mecanismos de priorización de vulnerabilidades basados en su criticidad técnica y en su nivel de explotación real.
- Desarrollar una capa de backend utilizando tecnologías basadas en Java que permita gestionar la lógica del sistema y exponer una API para el acceso a la información. (Véase Anexo E)
- Implementar una interfaz frontend que permita visualizar las vulnerabilidades procesadas, facilitando la interacción del usuario con el sistema. (Véase Anexo G)
- Automatizar el proceso de generación de alertas mediante herramientas de comunicación, permitiendo notificaciones en tiempo real.
- Diseñar un sistema de almacenamiento estructurado que permita registrar, consultar y analizar la información procesada. (Véase Anexo B)
- Incorporar métricas que permitan evaluar el desempeño del sistema, tales como tiempos de respuesta (MTTR) y volumen de vulnerabilidades procesadas.
- Mejorar la trazabilidad de los procesos de gestión de vulnerabilidades, facilitando su seguimiento y auditoría.

5. ALCANCES Y LIMITACIONES

5.1. ALCANCES

El presente trabajo se enfoca en el desarrollo de un sistema automatizado orientado al monitoreo, análisis, priorización y notificación de vulnerabilidades de seguridad. La solución propuesta permite integrar información proveniente de fuentes externas, tales como bases de datos de vulnerabilidades y catálogos de explotación activa, con información interna relacionada a los activos de una organización.

El sistema implementado contempla la automatización de un pipeline de procesamiento que incluye la recolección de datos, su análisis mediante reglas definidas, la correlación con activos internos y la generación de notificaciones en tiempo real. Asimismo, incorpora mecanismos de almacenamiento que permiten registrar la información procesada y facilitar su consulta posterior.

La solución se desarrolla utilizando herramientas de automatización de workflows, bases de datos relacionales y entornos contenedorizados, lo que permite su ejecución en entornos de laboratorio y su posible adaptación a contextos organizacionales reales.

Este trabajo se centra en la mejora del proceso de gestión de vulnerabilidades desde una perspectiva operativa, priorizando la eficiencia en el análisis y la reducción de tiempos de respuesta.

5.2. LIMITACIONES

El sistema desarrollado presenta ciertas limitaciones que deben ser consideradas. En primer lugar, la solución depende de la disponibilidad y calidad de la información proporcionada por fuentes externas, por lo que eventuales inconsistencias o demoras en dichas fuentes pueden impactar en los resultados obtenidos.

En segundo lugar, el sistema se implementa en un entorno de laboratorio, lo que implica que no contempla la totalidad de escenarios posibles en infraestructuras reales de gran escala. Por lo tanto, su aplicación en entornos productivos requeriría ajustes adicionales, especialmente en aspectos relacionados con seguridad, escalabilidad y gestión de accesos.

Asimismo, el proceso de priorización se basa en criterios predefinidos y reglas configuradas, lo que puede no cubrir todos los casos posibles o adaptarse dinámicamente a cambios en el contexto de amenazas.

Finalmente, el sistema no implementa mecanismos automáticos de remediación sobre los sistemas afectados, limitándose a la detección, análisis y notificación de vulnerabilidades, dejando la acción correctiva a los responsables correspondientes.

MARCO TEÓRICO

El presente capítulo tiene como objetivo establecer los fundamentos conceptuales que sustentan el desarrollo del sistema propuesto. Para ello, se abordan los principales conceptos relacionados con la seguridad informática, la gestión de vulnerabilidades y las herramientas utilizadas para su análisis y priorización.

1. SEGURIDAD INFORMÁTICA

La seguridad informática comprende el conjunto de prácticas, tecnologías y procesos orientados a proteger los sistemas de información frente a accesos no autorizados, alteraciones indebidas o interrupciones en su funcionamiento. Su objetivo principal es garantizar la confidencialidad, integridad y disponibilidad de la información, principios conocidos como la tríada CIA.

En entornos organizacionales, la seguridad informática se ha convertido en un componente esencial debido al aumento de amenazas y a la dependencia creciente de sistemas digitales. La gestión adecuada de la seguridad implica no solo la implementación de controles técnicos, sino también la adopción de procesos que permitan identificar, evaluar y mitigar riesgos de manera continua.

2 AMENAZAS Y VULNERABILIDADES

En el contexto de la seguridad informática, una **amenaza** se define como cualquier evento potencial que puede causar daño a un sistema o comprometer la información que este maneja. Las amenazas pueden originarse tanto en actores externos, como atacantes, como en factores internos, tales como errores humanos o fallas de configuración.

Por su parte, una **vulnerabilidad** corresponde a una debilidad o fallo en un sistema que puede ser explotado por una amenaza para materializar un ataque. La existencia de vulnerabilidades no gestionadas representa uno de los principales riesgos en entornos tecnológicos, ya que constituyen puntos de entrada para posibles incidentes de seguridad.

La **gestión de vulnerabilidades** implica la identificación, análisis, priorización y mitigación de estas debilidades. Este proceso se vuelve especialmente complejo debido al gran volumen de vulnerabilidades reportadas y a la necesidad de determinar cuáles requieren atención inmediata.

3. MODELOS DE SEGURIDAD

Los modelos de seguridad constituyen marcos conceptuales que estructuran la protección de sistemas de información mediante la identificación, evaluación y mitigación sistemática de riesgos. En el ámbito de la ciberseguridad organizacional, estos marcos evolucionaron desde enfoques puramente reactivos hacia modelos de gestión continua y proactiva.

ISO/IEC 27001 y la gestión de riesgos de seguridad. El estándar internacional ISO/IEC 27001 establece los requisitos para implementar, mantener y mejorar un Sistema de Gestión de Seguridad de la Información (SGSI). Dentro de este marco, la gestión de vulnerabilidades es un control explícito (Anexo A, control 8.8: *Gestión de las vulnerabilidades técnicas*), que exige identificar, evaluar y tratar oportunamente las debilidades técnicas de los activos de información. Este estándar sitúa la gestión de vulnerabilidades dentro de un ciclo de mejora continua (Plan-Do-Check-Act), subrayando que no se trata de una tarea puntual sino de un proceso permanente.

NIST Cybersecurity Framework (CSF). El Marco de Ciberseguridad del NIST organiza las capacidades de seguridad en cinco funciones: Identificar, Proteger, Detectar, Responder y Recuperar. La gestión de vulnerabilidades atraviesa principalmente las funciones de Identificación (ID.RA: evaluación de riesgos sobre activos) y Detección (DE.CM: monitoreo continuo). El sistema propuesto en este trabajo se alinea explícitamente con estas funciones: el pipeline automatizado implementa monitoreo continuo (DE.CM-8: análisis de vulnerabilidades) y la correlación con activos internos implementa la evaluación de riesgos en contexto (ID.RA-1).

OWASP y la gestión de vulnerabilidades en dependencias de software. El Open Web Application Security Project (OWASP) incorpora la gestión de componentes vulnerables como una de las diez categorías de riesgo más críticas en su edición 2021 (A06:2021 – Vulnerable and Outdated Components). Esta categoría, que ascendió desde la posición novena en 2017 hasta la sexta en 2021, refleja el crecimiento del riesgo asociado al uso de dependencias de software con vulnerabilidades conocidas. El sistema propuesto integra fuentes como GitHub Advisory precisamente para abordar este vector, complementando la perspectiva de OWASP con mecanismos automatizados de detección.

Limitaciones del modelo CVSS-only y enfoques contemporáneos. El sistema Common Vulnerability Scoring System, en sus versiones 3.1 y 4.0, provee métricas estandarizadas de severidad técnica pero presenta limitaciones documentadas en la literatura. Jacobs et al.

(2021) demostraron que la severidad CVSS tiene baja correlación con la probabilidad real de explotación, dado que la gran mayoría de vulnerabilidades catalogadas nunca son explotadas activamente. En consecuencia, organizaciones que priorizan exclusivamente por CVSS incurren en una asignación ineficiente de recursos, atendiendo vulnerabilidades teóricamente críticas mientras vulnerabilidades de menor puntuación pero activamente explotadas permanecen sin gestionar.

Ante esta limitación, el enfoque contemporáneo combina la severidad técnica con indicadores de explotación activa. El catálogo KEV de CISA representa la operacionalización institucional de este enfoque: una vulnerabilidad presente en KEV ha sido confirmada como explotada en entornos reales, lo que la convierte en una amenaza inmediata independientemente de su puntuación CVSS. Esta diferenciación entre riesgo técnico y riesgo operativo constituye el núcleo conceptual del modelo de priorización implementado en el presente trabajo.

Arquitectura en capas para sistemas de gestión de seguridad. La adopción de una arquitectura orientada a servicios con separación entre capa de datos, lógica de negocio e interfaz de usuario responde a principios de diseño ampliamente validados en sistemas de información. Este modelo facilita la evolución independiente de componentes, habilita la integración con herramientas externas mediante APIs estandarizadas y mejora la mantenibilidad del sistema. En el contexto específico de plataformas de gestión de seguridad, Sinha (2024) señala que la modularidad arquitectónica es un factor crítico para la escalabilidad de soluciones SOAR (*Security Orchestration, Automation and Response*) en entornos de nube, lo cual es directamente aplicable al diseño propuesto.

4. HERRAMIENTAS DE ANÁLISIS

El análisis de vulnerabilidades se apoya en diversas herramientas y fuentes de información que permiten identificar, describir y evaluar debilidades en sistemas informáticos. Estas fuentes proporcionan datos estructurados que son fundamentales para la automatización de procesos de monitoreo y priorización.

Una de las principales referencias en este ámbito es el sistema Common Vulnerabilities and Exposures (CVE), desarrollado y mantenido por la organización MITRE. El CVE proporciona un identificador único para cada vulnerabilidad conocida públicamente,

siguiendo un formato estandarizado (por ejemplo, CVE-2024-12345). Este sistema permite unificar la nomenclatura utilizada en la industria, facilitando la interoperabilidad entre distintas herramientas, bases de datos y soluciones de seguridad.

Cada entrada CVE incluye una descripción general de la vulnerabilidad, referencias asociadas y metadatos básicos. Sin embargo, el sistema CVE no proporciona por sí mismo una evaluación detallada del impacto o severidad de la vulnerabilidad, sino que actúa como un identificador común que permite correlacionar información entre distintas fuentes.

En este sentido, la National Vulnerability Database (NVD), gestionada por el National Institute of Standards and Technology (NIST), extiende la información disponible en los registros CVE, incorporando métricas adicionales como el puntaje de severidad basado en el estándar CVSS, vectores de ataque y datos enriquecidos que permiten un análisis más profundo. La NVD constituye, por lo tanto, una fuente clave para la evaluación técnica del riesgo asociado a una vulnerabilidad.

No obstante, tanto CVE como NVD presentan una limitación relevante: su enfoque está centrado en la severidad potencial de las vulnerabilidades, sin considerar necesariamente si estas están siendo explotadas activamente en entornos reales. Para abordar esta limitación, se incorpora el catálogo Known Exploited Vulnerabilities (KEV), mantenido por la Cybersecurity and Infrastructure Security Agency (CISA).

El catálogo KEV identifica vulnerabilidades que han sido confirmadas como explotadas activamente, proporcionando un enfoque orientado al riesgo real. Esta información resulta especialmente relevante para la priorización, ya que permite distinguir entre vulnerabilidades teóricamente críticas y aquellas que representan una amenaza inmediata.

Adicionalmente, en el marco de este trabajo se utiliza la base de datos GitHub Advisory Database, que proporciona información sobre vulnerabilidades en software y dependencias ampliamente utilizadas. Esta fuente permite complementar la información proveniente de CVE y NVD, incorporando datos específicos del ecosistema de desarrollo y facilitando la identificación de vulnerabilidades en componentes de software reales.

La integración de estas fuentes —CVE, NVD, CISA KEV y GitHub Advisory— permite construir una visión más completa del panorama de vulnerabilidades, combinando información sobre identificación, severidad técnica y explotación activa. En este proyecto, dichos datos son procesados mediante un sistema automatizado que permite

correlacionarlos con activos internos y priorizar las vulnerabilidades en función de su impacto potencial.

La utilización de herramientas de automatización de workflows, como n8n, posibilita la implementación de este proceso de manera eficiente, reduciendo la dependencia de intervenciones manuales y mejorando los tiempos de respuesta en la gestión de vulnerabilidades.

MARCO METODOLÓGICO

El presente capítulo describe el enfoque metodológico adoptado para el desarrollo del sistema propuesto, detallando el tipo de investigación, el diseño aplicado, las técnicas de recolección de datos y los instrumentos utilizados.

La metodología seleccionada tiene como objetivo garantizar la coherencia entre los objetivos planteados y la solución implementada, permitiendo validar el funcionamiento del sistema en un entorno controlado y analizar su desempeño en términos de eficiencia operativa y reducción de tiempos de respuesta.

1. TIPO DE INVESTIGACIÓN

El presente trabajo se enmarca dentro de una investigación de tipo aplicada, ya que tiene como finalidad el desarrollo de una solución tecnológica orientada a resolver un problema concreto en el ámbito de la ciberseguridad, específicamente en la gestión de vulnerabilidades. Este tipo de investigación se caracteriza por la generación de conocimiento con un propósito práctico, buscando implementar soluciones funcionales que puedan ser utilizadas en contextos reales.

Asimismo, se adopta un enfoque descriptivo, dado que se analizan las características, comportamiento y tratamiento de las vulnerabilidades dentro de un sistema automatizado, permitiendo comprender su impacto en los procesos de gestión de seguridad.

Adicionalmente, el trabajo incorpora un enfoque experimental, en tanto se diseña, implementa y evalúa un sistema en un entorno controlado, con el propósito de validar su funcionamiento y analizar su desempeño. Este enfoque permite observar el comportamiento del sistema frente a distintos escenarios y medir su efectividad en la mejora de los procesos de gestión de vulnerabilidades.

La combinación de estos enfoques resulta adecuada para el presente estudio, ya que permite abordar el problema desde una perspectiva integral, contemplando tanto su análisis conceptual como la implementación y validación de una solución concreta.

2. DISEÑO DE INVESTIGACIÓN

El diseño de la investigación se fundamenta en la construcción e implementación de un sistema automatizado orientado a la gestión de vulnerabilidades, integrando múltiples fuentes de información y aplicando un enfoque basado en procesamiento de datos mediante workflows.

El sistema propuesto se estructura como un pipeline de procesamiento que simula el funcionamiento de un entorno real de gestión de vulnerabilidades. Este pipeline está compuesto por una serie de etapas interrelacionadas que permiten transformar datos provenientes de fuentes externas en información útil para la toma de decisiones.

Las etapas principales del sistema incluyen:

- Recolección automatizada de datos desde fuentes externas de vulnerabilidades.
- Procesamiento, normalización y filtrado de la información obtenida.
- Correlación de vulnerabilidades con activos internos.
- Aplicación de criterios de priorización basados en riesgo.
- Generación de notificaciones automáticas para la gestión de incidentes.

Este diseño permite evaluar el sistema desde una perspectiva integral, considerando tanto su funcionamiento técnico como su impacto en los procesos de gestión de seguridad. Asimismo, facilita la medición de indicadores de desempeño, tales como los tiempos de respuesta (MTTR) y el volumen de vulnerabilidades procesadas, lo que permite validar la efectividad de la solución propuesta.

El enfoque adoptado se alinea con prácticas actuales de automatización en ciberseguridad, permitiendo reproducir un flujo de trabajo similar al utilizado en entornos reales, pero dentro de un contexto controlado que facilita su análisis y evaluación.

3. TÉCNICAS DE RECOLECCIÓN DE DATOS

La estrategia metodológica del presente trabajo diferencia dos planos de recolección de datos: el plano operativo del sistema y el plano evaluativo de la investigación.

En el plano operativo, el sistema recolecta información mediante consultas automatizadas a APIs externas (GitHub Advisory Database y CISA KEV) ejecutadas por workflows en n8n. Esta recolección es continua, programada y normalizada antes de su almacenamiento en PostgreSQL.

En el plano evaluativo, la investigación emplea tres técnicas para validar el desempeño del sistema:

Medición de métricas de desempeño operativo. Se registran indicadores cuantitativos durante las ejecuciones de prueba del sistema, incluyendo: tiempo total de procesamiento del pipeline, volumen de vulnerabilidades procesadas, tasa de correlación positiva con activos internos y distribución de prioridades asignadas.

Análisis de trazas de ejecución. Se analizan los registros almacenados en la tabla `vulnerabilities_audit` para verificar la correcta aplicación de los criterios de priorización, detectar inconsistencias en el procesamiento y validar la integridad del flujo de datos extremo a extremo.

Comparación con escenario base. Se establece un escenario de referencia que simula el proceso manual de revisión de vulnerabilidades, estimando el tiempo que demandaría un analista para procesar manualmente el mismo volumen de datos que el sistema gestiona automáticamente. Esta comparación permite cuantificar la mejora en el MTTR.

4. INSTRUMENTOS

Para la implementación del sistema propuesto se utilizaron diversos instrumentos tecnológicos que permiten llevar a cabo las tareas de recolección, procesamiento, almacenamiento, gestión y visualización de la información.

En primer lugar, se empleó una herramienta de **automatización de workflows (n8n)**, la cual constituye el núcleo del sistema. Esta herramienta permite diseñar y ejecutar flujos de procesamiento de datos de manera visual, facilitando la integración de múltiples fuentes de información, la aplicación de reglas de procesamiento y la automatización de tareas.

En segundo lugar, se utilizó una **base de datos relacional (PostgreSQL)** para el almacenamiento estructurado de la información recolectada y procesada. Este componente

permite gestionar los datos de vulnerabilidades, registrar eventos, almacenar resultados del análisis y facilitar su consulta mediante distintos componentes del sistema.

Asimismo, el sistema se ejecuta en un entorno contenedorizado mediante **Docker**, lo que garantiza la portabilidad, reproducibilidad y aislamiento de los distintos servicios que conforman la solución.

Como parte de la arquitectura del sistema, se incorpora una capa de **backend** desarrollada en **Java**, utilizando un framework orientado a la construcción de aplicaciones web (**Spring Boot**). Esta capa se encarga de gestionar la lógica del sistema, interactuar con la base de datos y exponer una API REST que permite el acceso a la información procesada. De esta manera, se desacopla la lógica de negocio del procesamiento automatizado y de la interfaz de usuario.

Adicionalmente, se implementa una **interfaz frontend (DEFINIR)** que permite visualizar la información de vulnerabilidades, consultar datos relevantes y facilitar la interacción del usuario con el sistema. Esta capa contribuye a mejorar la usabilidad de la solución y permite transformar los datos procesados en información accesible.

Para la lógica de procesamiento, se utilizan scripts y funciones en **JavaScript** que permiten transformar los datos obtenidos, aplicar criterios de priorización y estructurar la información. Estos componentes son utilizados principalmente dentro de los workflows automatizados.

Por último, se integran herramientas de comunicación, como **Slack**, para la notificación automática de vulnerabilidades detectadas, permitiendo alertar a los usuarios en tiempo real y mejorar la capacidad de respuesta ante incidentes.

En conjunto, estos instrumentos permiten implementar un sistema integral, modular y escalable para la gestión automatizada de vulnerabilidades, alineado con los objetivos planteados en el presente trabajo.

DESARROLLO DE ESTUDIO

En el presente capítulo se describe el desarrollo e implementación del sistema propuesto, detallando su arquitectura, las herramientas utilizadas y el procedimiento seguido para su construcción y validación. El sistema se diseña con un enfoque modular basado en una arquitectura en capas, permitiendo separar responsabilidades y facilitar su mantenimiento, escalabilidad y evolución.

1. ARQUITECTURA DEL SISTEMA

1.1 ARQUITECTURA GENERAL

La arquitectura del sistema se basa en un enfoque en capas que organiza los distintos componentes según su función dentro del proceso de gestión de vulnerabilidades.

El sistema se compone de las siguientes capas principales:

- Capa de ingesta y procesamiento: encargada de la recolección automatizada de datos desde fuentes externas mediante workflows implementados en n8n. En esta capa se realiza el filtrado, transformación y normalización de la información.
- Capa de almacenamiento: implementada mediante una base de datos relacional (PostgreSQL), donde se persisten los datos de vulnerabilidades, activos y resultados del análisis.
- Capa de lógica de negocio (backend): desarrollada en Java, encargada de gestionar el acceso a los datos, aplicar reglas adicionales y exponer funcionalidades mediante una API REST.
- Capa de presentación (frontend): permite la visualización de la información y la interacción del usuario con el sistema.

Esta arquitectura permite desacoplar los distintos componentes, facilitando la evolución independiente de cada capa y alineándose con las buenas prácticas actuales en el desarrollo de sistemas.

1.2 MODELO DE DATOS

Dentro de la capa de almacenamiento, se define una estructura de base de datos orientada a la gestión de vulnerabilidades.

Las principales entidades del sistema son:

- **Vulnerabilities:** almacena información sobre vulnerabilidades detectadas, incluyendo identificador CVE, severidad, descripción y estado.
- **Assets:** representa los activos de la organización que pueden verse afectados.
- **Vulnerability_Asset:** tabla intermedia que permite relacionar vulnerabilidades con activos, facilitando la correlación entre amenazas y contexto organizacional.

Este modelo permite representar de manera estructurada la relación entre vulnerabilidades y activos, posibilitando su análisis y priorización en función del impacto.

2 HERRAMIENTAS UTILIZADAS

El desarrollo del sistema propuesto requirió la utilización de diversas herramientas tecnológicas, seleccionadas en función de su capacidad para integrarse entre sí, su flexibilidad y su adecuación a los objetivos planteados. Cada una de estas herramientas cumple un rol específico dentro de la arquitectura del sistema, contribuyendo a la implementación de una solución integral para la gestión de vulnerabilidades.

2.1 AUTOMATIZACIÓN DE WORKFLOWS

Para la automatización de los procesos de recolección, procesamiento y orquestación de datos se utilizó la herramienta n8n, la cual permite diseñar flujos de trabajo mediante un enfoque visual basado en nodos interconectados.

Esta herramienta constituye el núcleo del sistema, ya que permite integrar múltiples fuentes de información externas —como bases de datos de vulnerabilidades— y ejecutar de manera automatizada las distintas etapas del procesamiento, incluyendo la transformación, filtrado y validación de datos.

El uso de workflows automatizados permite reducir significativamente la intervención manual, mejorar la consistencia en el tratamiento de la información y garantizar la ejecución periódica de tareas críticas para el monitoreo continuo de vulnerabilidades. Asimismo, n8n facilita la implementación de lógica condicional y la integración con servicios externos, lo que resulta fundamental para la orquestación del sistema.

2.2 BASE DE DATOS

El almacenamiento de la información se implementa mediante PostgreSQL, una base de datos relacional que permite gestionar datos de forma estructurada, segura y eficiente.

Este componente cumple un rol central dentro del sistema, ya que actúa como repositorio persistente de la información recolectada y procesada. En la base de datos se almacenan tanto los datos provenientes de fuentes externas como la información generada durante el procesamiento, incluyendo la relación entre vulnerabilidades y activos.

La elección de un modelo relacional permite representar de manera consistente las relaciones entre entidades, facilitar la ejecución de consultas complejas y garantizar la integridad de los datos. Asimismo, PostgreSQL ofrece un alto nivel de confiabilidad y escalabilidad, lo que lo convierte en una opción adecuada para este tipo de sistemas.

2.3 BACKEND

La capa de backend se implementa utilizando Java mediante el framework Spring Boot, el cual permite el desarrollo de aplicaciones robustas y escalables orientadas a servicios.

El backend cumple un rol fundamental dentro de la arquitectura del sistema, actuando como intermediario entre la base de datos, los procesos automatizados y la interfaz de usuario. A través de esta capa se centraliza la lógica de negocio, se gestionan las operaciones sobre los datos y se expone una API REST que permite el acceso a la información procesada.

Entre las principales funciones del backend se destacan:

- Gestión del acceso y manipulación de datos almacenados en la base de datos.
- Exposición de endpoints para la consulta y actualización de información.
- Implementación de lógica de filtrado, validación y control de estados.
- Integración con otros componentes del sistema.

Los principales endpoints implementados incluyen:

- GET /vulnerabilities: obtención de vulnerabilidades procesadas.
- GET /vulnerabilities/{id}: consulta detallada de una vulnerabilidad específica.

- GET /assets: obtención de los activos registrados en el sistema.
- PUT /vulnerabilities/{id}/status: actualización del estado de una vulnerabilidad.

Este enfoque permite desacoplar la lógica de negocio del resto de los componentes, favoreciendo la mantenibilidad del sistema y habilitando su extensión futura.

2.4 FRONTEND

La capa de frontend tiene como objetivo proporcionar una interfaz de usuario que permita visualizar de manera clara y estructurada la información procesada por el sistema.

A través de esta interfaz, los usuarios pueden acceder a datos relevantes sobre vulnerabilidades, consultar su estado, analizar su nivel de criticidad y explorar la información almacenada en el sistema. Esta capa transforma los datos en información accesible, facilitando la toma de decisiones.

La incorporación de un frontend mejora la usabilidad del sistema y permite su aplicación en contextos reales, donde resulta necesario contar con herramientas visuales que simplifiquen la interacción con grandes volúmenes de información.

2.5 INFRAESTRUCTURA

El sistema se ejecuta en un entorno contenedorizado mediante Docker, lo que permite encapsular los distintos componentes en unidades independientes de ejecución.

Este enfoque facilita la portabilidad del sistema, ya que permite replicar el entorno de ejecución en diferentes plataformas sin necesidad de configuraciones adicionales. Asimismo, garantiza la reproducibilidad del sistema y simplifica su despliegue, lo cual resulta especialmente relevante en entornos de desarrollo y pruebas.

2.6 NOTIFICACIONES

Para la comunicación de eventos relevantes se integran herramientas de mensajería como Slack, las cuales permiten la generación de notificaciones automáticas ante la detección de vulnerabilidades.

Estas notificaciones se generan a partir de los workflows automatizados y permiten alertar a los usuarios en tiempo real, mejorando la capacidad de respuesta ante incidentes de seguridad.

La integración con herramientas de comunicación contribuye a cerrar el ciclo de gestión de vulnerabilidades, permitiendo que la información procesada no solo sea almacenada, sino también comunicada de manera efectiva a los responsables.

RESULTADOS

AGREGAR: números concretos (ej: “MTTR de 30s”), tablas o gráficos, ejemplo real de vulnerabilidad procesada

En el presente capítulo se presentan los resultados obtenidos a partir de la implementación del sistema propuesto, evaluando su funcionamiento en un entorno controlado y analizando su desempeño en relación con los objetivos planteados.

La evaluación se centra en verificar la correcta integración de los componentes del sistema, la automatización de los procesos de gestión de vulnerabilidades y la mejora en los tiempos de respuesta frente a eventos de seguridad.

1. FUNCIONALIDADES IMPLEMENTADAS

El sistema desarrollado permite automatizar de manera integral el proceso de gestión de vulnerabilidades, incorporando múltiples funcionalidades que abarcan desde la recolección de datos hasta la visualización de la información.

Entre las principales funcionalidades implementadas se destacan:

- Recolección automatizada de información desde fuentes externas, incluyendo bases de datos de vulnerabilidades y catálogos de explotación activa.
- Procesamiento, filtrado y normalización de los datos mediante workflows automatizados.
- Correlación de vulnerabilidades con activos internos registrados en el sistema.
- Aplicación de criterios de priorización basados en severidad técnica y contexto operativo.
- Almacenamiento estructurado de la información en una base de datos relacional.
- Exposición de una API REST mediante la capa de backend para la consulta y gestión de los datos.
- Visualización de la información a través de una interfaz frontend, permitiendo la interacción con el sistema.
- Generación de notificaciones automáticas a través de herramientas de comunicación, alertando sobre vulnerabilidades relevantes.

Estas funcionalidades permiten cubrir el ciclo completo de gestión de vulnerabilidades, desde la detección hasta la comunicación de eventos.

2. EVALUACIÓN DEL SISTEMA

La evaluación del sistema se realizó en un entorno de prueba, simulando el flujo de procesamiento de vulnerabilidades mediante la ejecución de workflows automatizados.

Se verificó la correcta integración entre los distintos componentes del sistema, incluyendo la recolección de datos, su procesamiento, almacenamiento, consulta a través del backend y visualización mediante el frontend.

Asimismo, se evaluó la capacidad del sistema para operar de manera continua, procesando información de forma automatizada y generando resultados consistentes.

Los resultados obtenidos evidencian que el sistema funciona de manera estable, permitiendo automatizar tareas que tradicionalmente se realizan de forma manual, lo que contribuye a mejorar la eficiencia operativa.

3. MÉTRICAS OBTENIDAS

Con el objetivo de evaluar el desempeño del sistema, se definieron y analizaron diversas métricas relacionadas con la gestión de vulnerabilidades.

Entre las principales métricas consideradas se encuentran:

- Tiempo medio de respuesta (MTTR): se evaluó la reducción del tiempo transcurrido entre la detección de una vulnerabilidad y la generación de una notificación. La automatización del proceso permitió reducir significativamente este tiempo en comparación con un enfoque manual.
- Volumen de vulnerabilidades procesadas: el sistema demostró capacidad para procesar múltiples registros de manera automatizada, manteniendo consistencia en los resultados.
- Tasa de automatización: se observó que la mayoría de las etapas del proceso de gestión de vulnerabilidades se ejecutan sin intervención manual, lo que evidencia un alto grado de automatización.
- Tiempo de procesamiento: el sistema permite procesar y almacenar información en tiempos reducidos, favoreciendo la disponibilidad de datos para su consulta.

Estas métricas permiten validar la eficiencia del sistema y su capacidad para mejorar los procesos de gestión de vulnerabilidades.

4. VALIDACIÓN DE OBJETIVOS

Los resultados obtenidos permiten validar el cumplimiento de los objetivos planteados en el presente trabajo.

En relación con el objetivo general, se logró desarrollar un sistema integral que automatiza la detección, análisis, priorización y notificación de vulnerabilidades, integrando múltiples fuentes de información y reduciendo la dependencia de procesos manuales.

En cuanto a los objetivos específicos:

- Se implementó un pipeline automatizado para la recolección y procesamiento de datos.
- Se logró integrar información de distintas fuentes y correlacionarla con activos internos.
- Se desarrolló un mecanismo de priorización basado en múltiples criterios.
- Se implementó una API backend que permite el acceso y gestión de la información.
- Se desarrolló una interfaz frontend que facilita la visualización de los datos.
- Se automatizó la generación de notificaciones en tiempo real.
- Se incorporaron métricas que permiten evaluar el desempeño del sistema.

En conjunto, estos resultados evidencian que el sistema cumple con los objetivos propuestos y constituye una solución funcional para la gestión automatizada de vulnerabilidades.

DISCUSIÓN

Los resultados obtenidos a partir de la implementación del sistema propuesto permiten analizar el impacto de la automatización en los procesos de gestión de vulnerabilidades, evidenciando mejoras significativas en términos de eficiencia operativa, reducción de tiempos de respuesta y capacidad de procesamiento de información.

En primer lugar, la automatización del pipeline de procesamiento demostró ser un factor determinante en la optimización del flujo de trabajo. La integración de múltiples fuentes de información, combinada con la ejecución automatizada de tareas, permitió reducir la dependencia de intervenciones manuales, minimizando errores humanos y mejorando la consistencia en el tratamiento de los datos.

Asimismo, la incorporación de mecanismos de priorización basados en múltiples criterios —como la severidad técnica y el contexto operativo— permitió superar las limitaciones de los enfoques tradicionales centrados exclusivamente en métricas como CVSS. Este enfoque posibilita una evaluación más realista del riesgo, alineándose con prácticas actuales en ciberseguridad que priorizan el impacto efectivo sobre el entorno.

Desde el punto de vista arquitectónico, la adopción de un modelo en capas, que separa la lógica de procesamiento, la lógica de negocio y la interfaz de usuario, contribuyó a mejorar la organización del sistema y su mantenibilidad. La incorporación de una capa de backend desarrollada en Java, junto con una interfaz frontend, permitió transformar un sistema inicialmente centrado en la automatización en una solución más completa, orientada a la interacción con el usuario.

En este sentido, la implementación de una API REST no solo facilitó el acceso a la información, sino que también permitió desacoplar los distintos componentes del sistema, favoreciendo su escalabilidad y posibilitando su integración con otros sistemas en el futuro.

Por otro lado, la utilización de herramientas como n8n para la automatización de workflows evidenció ser una alternativa eficiente para la orquestación de procesos, permitiendo implementar lógica compleja sin necesidad de desarrollar soluciones desde cero. Esto resulta particularmente relevante en contextos donde se requiere flexibilidad y rapidez en la implementación.

En cuanto a la capa de almacenamiento, el uso de una base de datos relacional permitió estructurar la información de manera consistente, facilitando su consulta y análisis.

La modelización de entidades como vulnerabilidades y activos resultó clave para contextualizar la información y mejorar la calidad del análisis.

Desde una perspectiva aplicada, el sistema desarrollado presenta características que lo hacen viable para su utilización en entornos organizacionales reales. La integración de notificaciones automáticas, la disponibilidad de una interfaz de usuario y la posibilidad de consulta mediante API permiten su adaptación a distintos escenarios de uso.

No obstante, es importante considerar ciertas limitaciones. El sistema se evaluó en un entorno controlado, por lo que su implementación en entornos productivos podría requerir ajustes adicionales en aspectos como seguridad, escalabilidad y gestión de accesos. Asimismo, los criterios de priorización utilizados, si bien son efectivos, podrían enriquecerse mediante la incorporación de fuentes adicionales de inteligencia de amenazas o técnicas más avanzadas de análisis.

En este contexto, el sistema desarrollado constituye una base sólida sobre la cual pueden incorporarse mejoras futuras, tales como la integración de modelos de análisis más avanzados, la ampliación de fuentes de datos o la automatización de acciones de respuesta.

En síntesis, los resultados obtenidos y su análisis permiten afirmar que la automatización de la gestión de vulnerabilidades, combinada con una arquitectura modular y el uso de herramientas adecuadas, contribuye significativamente a mejorar la eficiencia y efectividad de los procesos de seguridad, alineándose con las tendencias actuales en el ámbito de la ciberseguridad.

CONCLUSIONES

Los resultados obtenidos a partir de la implementación del sistema propuesto permiten analizar el impacto de la automatización en los procesos de gestión de vulnerabilidades, evidenciando mejoras significativas en términos de eficiencia operativa, reducción de tiempos de respuesta y capacidad de procesamiento de información.

En primer lugar, la automatización del pipeline de procesamiento demostró ser un factor determinante en la optimización del flujo de trabajo. La integración de múltiples fuentes de información, combinada con la ejecución automatizada de tareas, permitió reducir la dependencia de intervenciones manuales, minimizando errores humanos y mejorando la consistencia en el tratamiento de los datos.

Asimismo, la incorporación de mecanismos de priorización basados en múltiples criterios —como la severidad técnica y el contexto operativo— permitió superar las limitaciones de los enfoques tradicionales centrados exclusivamente en métricas como CVSS. Este enfoque posibilita una evaluación más realista del riesgo, alineándose con prácticas actuales en ciberseguridad que priorizan el impacto efectivo sobre el entorno.

Desde el punto de vista arquitectónico, la adopción de un modelo en capas, que separa la lógica de procesamiento, la lógica de negocio y la interfaz de usuario, contribuyó a mejorar la organización del sistema y su mantenibilidad. La incorporación de una capa de backend desarrollada en Java, junto con una interfaz frontend, permitió transformar un sistema inicialmente centrado en la automatización en una solución más completa, orientada a la interacción con el usuario.

En este sentido, la implementación de una API REST no solo facilitó el acceso a la información, sino que también permitió desacoplar los distintos componentes del sistema, favoreciendo su escalabilidad y posibilitando su integración con otros sistemas en el futuro.

Por otro lado, la utilización de herramientas como n8n para la automatización de workflows evidenció ser una alternativa eficiente para la orquestación de procesos, permitiendo implementar lógica compleja sin necesidad de desarrollar soluciones desde cero. Esto resulta particularmente relevante en contextos donde se requiere flexibilidad y rapidez en la implementación.

En cuanto a la capa de almacenamiento, el uso de una base de datos relacional permitió estructurar la información de manera consistente, facilitando su consulta y análisis.

La modelización de entidades como vulnerabilidades y activos resultó clave para contextualizar la información y mejorar la calidad del análisis.

Desde una perspectiva aplicada, el sistema desarrollado presenta características que lo hacen viable para su utilización en entornos organizacionales reales. La integración de notificaciones automáticas, la disponibilidad de una interfaz de usuario y la posibilidad de consulta mediante API permiten su adaptación a distintos escenarios de uso.

No obstante, es importante considerar ciertas limitaciones. El sistema se evaluó en un entorno controlado, por lo que su implementación en entornos productivos podría requerir ajustes adicionales en aspectos como seguridad, escalabilidad y gestión de accesos. Asimismo, los criterios de priorización utilizados, si bien son efectivos, podrían enriquecerse mediante la incorporación de fuentes adicionales de inteligencia de amenazas o técnicas más avanzadas de análisis.

En este contexto, el sistema desarrollado constituye una base sólida sobre la cual pueden incorporarse mejoras futuras, tales como la integración de modelos de análisis más avanzados, la ampliación de fuentes de datos o la automatización de acciones de respuesta.

En síntesis, los resultados obtenidos y su análisis permiten afirmar que la automatización de la gestión de vulnerabilidades, combinada con una arquitectura modular y el uso de herramientas adecuadas, contribuye significativamente a mejorar la eficiencia y efectividad de los procesos de seguridad, alineándose con las tendencias actuales en el ámbito de la ciberseguridad.

RECOMENDACIONES

A partir del desarrollo y análisis del sistema propuesto, se identifican diversas oportunidades de mejora y líneas de evolución que podrían ser abordadas en futuras implementaciones o trabajos relacionados.

En primer lugar, se recomienda ampliar la integración con fuentes externas de información sobre vulnerabilidades, incorporando nuevas bases de datos y servicios de inteligencia de amenazas. Esto permitiría enriquecer el análisis y mejorar la precisión en la priorización de riesgos.

Asimismo, resulta conveniente profundizar los mecanismos de priorización mediante la incorporación de técnicas más avanzadas, como modelos de análisis basados en comportamiento o el uso de herramientas de inteligencia artificial que permitan identificar patrones y anomalías en los datos.

En relación con la arquitectura del sistema, se sugiere fortalecer los aspectos vinculados a la seguridad del backend, incluyendo mecanismos de autenticación, autorización y control de accesos. Esto resulta especialmente relevante en escenarios de despliegue en entornos productivos.

Por otro lado, se recomienda optimizar la escalabilidad del sistema, contemplando la posibilidad de manejar mayores volúmenes de datos y múltiples fuentes de información de manera simultánea. La implementación de arquitecturas distribuidas o el uso de servicios en la nube podrían contribuir a este objetivo.

En cuanto a la interfaz de usuario, se propone continuar desarrollando el frontend incorporando funcionalidades avanzadas de visualización, como dashboards interactivos, gráficos y filtros dinámicos, que permitan mejorar la experiencia del usuario y facilitar el análisis de la información.

Además, se sugiere avanzar hacia la automatización de acciones de respuesta ante vulnerabilidades, integrando mecanismos que permitan ejecutar medidas correctivas de manera controlada, como la generación de tickets, la aplicación de parches o la configuración de reglas de seguridad.

Desde una perspectiva organizacional, se recomienda integrar el sistema con otras herramientas de gestión utilizadas en entornos empresariales, como sistemas de gestión de

incidencias o plataformas de monitoreo, con el fin de consolidar la información y mejorar la coordinación entre equipos.

Finalmente, se destaca la importancia de realizar pruebas en entornos reales de producción, lo que permitiría validar el comportamiento del sistema en escenarios más complejos y evaluar su impacto en contextos organizacionales concretos.

En conjunto, estas recomendaciones buscan orientar la evolución del sistema hacia una solución más robusta, escalable y alineada con las necesidades actuales del ámbito de la ciberseguridad.

REFERENCIAS BIBLIOGRÁFICAS

- MITRE Corporation. (2024). *Common Vulnerabilities and Exposures (CVE)*. <https://cve.mitre.org/>
- National Institute of Standards and Technology (NIST). (2024). *National Vulnerability Database (NVD)*. <https://nvd.nist.gov/>
- Cybersecurity and Infrastructure Security Agency (CISA). (2024). *Known Exploited Vulnerabilities Catalog*. <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
- FIRST. (2023). *Common Vulnerability Scoring System v3.1: Specification Document*. <https://www.first.org/cvss/>
- GitHub. (2024). *GitHub Advisory Database*. <https://github.com/advisories>
- PostgreSQL Global Development Group. (2024). *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>
- Docker Inc. (2024). *Docker Documentation*. <https://docs.docker.com/>
- n8n GmbH. (2024). *n8n Documentation*. <https://docs.n8n.io/>
- Slack Technologies. (2024). *Slack API Documentation*. <https://api.slack.com/>
- Oracle. (2024). *Java Documentation*. <https://docs.oracle.com/en/java/>
- Pivotal Software. (2024). *Spring Boot Documentation*. <https://spring.io/projects/spring-boot>
- Ismail, M. (2025). Toward Robust Security Orchestration and Automated Incident Response Using Agentic-LLM. *Information (MDPI)*.
- Sinha, A. (2024). Challenges and best practices for integrating Security Orchestration, Automation, and Response (SOAR) platforms in cloud environments. *CEUR Workshop Proceedings*.
- Manzoor, J., et al. (2024). *Evaluating security and performance of open-source SIEM systems*. *PLOS ONE*.

ANEXOS

Los anexos presentados a continuación contienen información complementaria que respalda el desarrollo técnico del sistema propuesto, incluyendo estructuras de datos, configuraciones, flujos de trabajo y ejemplos de funcionamiento.

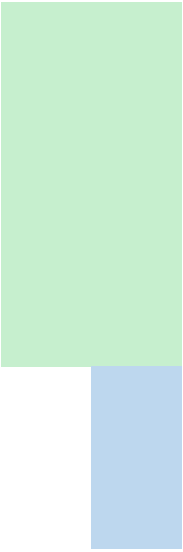
Anexo A — Cronograma de implementación

Este anexo presenta la planificación temporal del desarrollo del sistema, incluyendo las distintas etapas del proyecto y los tiempos estimados para cada una de ellas.

Tabla A1

Diagrama de Gantt del proyecto GEF Secure (diciembre 2025 – abril 2026)

#	Actividad	Responsable	Dic. 2025	Ene. 2026	Feb. 2026	Mar. 2026	Abr. 2026	P
1	Investigación y marco conceptual	Maricchiolo, G.						A
2	Diseño de arquitectura: pipeline, modelo de datos y componentes	Frankenberger, F.						A
3	Implementación del pipeline en n8n (15 nodos y lógica JS)	Frankenberger, F.						A
4	Integración con PostgreSQL (assets, vulnerabilities_audit, vulnerabilities_ignored)	Maricchiolo, G.						A
5	Desarrollo backend Java / Spring Boot y API REST	Frankenberger, F.						A

6	Integración Slack: notificaciones ante CRITICAL/HIGH	Barros Hernandez, M. E.		M
7	Implementación del frontend: visualización de vulnerabilidades	Barros Hernandez, M. E.		M
8	Pruebas, validación y medición de métricas (MTTR, automatización)	Equipo completo		A

Nota. Las celdas sombreadas en azul indican actividades de prioridad Alta (A); las celdas sombreadas en verde corresponden a actividades de prioridad Media (M). La columna P. indica el nivel de prioridad asignado. El solapamiento entre las actividades 4–5 y 6–7 responde a una estrategia de desarrollo incremental orientada a reducir el tiempo total de entrega. Elaboración propia (2026).

Tabla A2

Detalle de actividades, entregables y criterios de aceptación

#	Actividad	Inicio	Fin	Entregable	Criterio de aceptación
1	Investigación y marco conceptual	01/12/2025	15/01/2026	Documento de investigación / Marco Teórico	Marco teórico con referencias a estándares
2	Diseño de arquitectura: pipeline, modelo de datos y componentes	15/01/2026	07/02/2026	Diagrama de arquitectura en capas	Arquitectura en 4 capas documentada

3	Implementación del pipeline en n8n (15 nodos y lógica JS)	07/02/2026	28/02/2026	Workflow funcional en n8n	Pipeline ejecutable que clasifique CVEs por prioridad
4	Integración con PostgreSQL (assets, vulnerabilities_audit, vulnerabilities_ignored)	22/02/2026	14/03/2026	Base de datos configurada	Insertión, actualización y consulta desde pipeline
5	Desarrollo backend Java / Spring Boot y API REST	10/03/2026	31/03/2026	API REST con endpoints documentados	Endpoints responden correctamente; lógica desacoplada
6	Integración Slack: notificaciones ante CRITICAL/HIGH	28/03/2026	07/04/2026	Notificaciones operativas vía Slack	Mensaje con CVE, severidad y activo enviado automáticamente
7	Implementación del frontend: visualización de vulnerabilidades	28/03/2026	11/04/2026	Interfaz frontend funcional	Usuario consulta y filtra vulnerabilidades sin acceso directo a BD
8	Pruebas, validación y medición de métricas (MTTR, automatización)	11/04/2026	21/04/2026	Informe de resultados y métricas	Sistema end-to-end; métricas evidencian mejora

Nota. La columna Criterio de aceptación define la condición verificable que determina la finalización satisfactoria de cada actividad. Elaboración propia (2026).

Anexo B — Esquema de base de datos

El presente anexo describe el esquema de base de datos implementado en PostgreSQL para el sistema GEF Secure. El modelo contempla siete tablas principales y una vista auxiliar, organizadas para soportar el ciclo completo de gestión de vulnerabilidades: ingesta de datos, auditoría de hallazgos, gestión de descartes, almacenamiento de reportes, control de usuarios y registro de errores del sistema.

Para cada tabla se presenta: (a) una descripción funcional de su propósito dentro del sistema, (b) el script SQL de creación con sus restricciones e índices, y (c) una tabla descriptiva de sus campos principales. Al final del anexo se detallan las relaciones de clave foránea entre tablas.

B.1. Tabla environments

Define los entornos de infraestructura disponibles en el sistema (por ejemplo, Desarrollo, Testing y Producción). Cada activo registrado en el sistema pertenece a uno de estos entornos, lo que permite segmentar el análisis de vulnerabilidades según la criticidad operativa del contexto.

Script de creación:

```
CREATE TABLE IF NOT EXISTS public.environments (  
    id          BIGSERIAL PRIMARY KEY,  
    name        VARCHAR(50) NOT NULL UNIQUE,  
    business_criticality VARCHAR(20) NOT NULL,  
    description  TEXT,  
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Tabla B1*Descripción de campos de la tabla environments*

Campo	Tipo de dato	Descripción
id	BIGSERIAL	Clave primaria generada automáticamente.
name	VARCHAR(50)	Nombre del entorno (ej.: Producción). Valor único y obligatorio.
business_criticality	VARCHAR(20)	Nivel de criticidad de negocio asociado al entorno (ej.: ALTA, MEDIA, BAJA).
description	TEXT	Descripción libre del entorno. Campo opcional.
created_at	TIMESTAMP	Fecha y hora de creación del registro. Se asigna automáticamente.

Nota. Los tipos de dato corresponden a la nomenclatura de PostgreSQL 14. FK = Foreign Key. Elaboración propia (2026).

B.2. Tabla assets

Almacena el inventario de software detectado en cada entorno. Cada activo representa un componente de software instalado en un entorno específico, identificado por su nombre de paquete, ecosistema y versión. La tabla implementa una restricción de unicidad sobre la combinación (software, version, environment_id) para evitar duplicados.

Script de creación:

```

CREATE TABLE IF NOT EXISTS public.assets (
    id          BIGSERIAL PRIMARY KEY,
    name        VARCHAR(100),
    software     VARCHAR(255),
    ecosystem    VARCHAR(50),
    version      VARCHAR(50),
    last_scan    TIMESTAMP,
    description   TEXT,
    environment_id BIGINT,
    CONSTRAINT fk_assets_environment
    FOREIGN KEY (environment_id)
    REFERENCES public.environments (id) ON DELETE CASCADE,
    CONSTRAINT unique_asset_per_environment
    UNIQUE (software, version, environment_id)
);

```

Tabla B2

Descripción de campos de la tabla assets

Campo	Tipo de dato	Descripción
id	BIGSERIAL	Clave primaria generada automáticamente.
name	VARCHAR(100)	Nombre descriptivo del activo (ej.: nombre del servidor o aplicación).
software	VARCHAR(255)	Nombre del paquete o componente de software detectado.
ecosystem	VARCHAR(50)	Ecosistema del paquete (ej.: npm, pip, Maven).
version	VARCHAR(50)	Versión instalada del software.
last_scan	TIMESTAMP	Fecha y hora del último escaneo que detectó o actualizó este activo.
description	TEXT	Descripción adicional del activo. Campo opcional.
environment_id	BIGINT (FK)	Referencia al entorno al que pertenece el activo (FK → environments.id).

Nota. Los tipos de dato corresponden a la nomenclatura de PostgreSQL 14. FK = Foreign Key. Elaboración propia (2026).

B.3. Tabla scan_reports

Registra el historial de ejecuciones del workflow de escaneo. Cada fila representa una corrida completa del pipeline, almacenando métricas agregadas como el total de vulnerabilidades detectadas, auditadas e ignoradas, desglosadas por nivel de severidad. El campo environment_breakdown permite almacenar el detalle por entorno en formato JSON.

Script de creación:

```
CREATE TABLE IF NOT EXISTS public.scan_reports (
  id          BIGSERIAL PRIMARY KEY,
  executed_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  total_detected INTEGER NOT NULL,
  audited     INTEGER NOT NULL,
  ignored     INTEGER NOT NULL,
  criticals   INTEGER DEFAULT 0,
  highs      INTEGER DEFAULT 0,
  mediums    INTEGER DEFAULT 0,
  lows       INTEGER DEFAULT 0,
  system_status VARCHAR(50),
  report_message TEXT,
  environment_breakdown JSONB,
  target_type  VARCHAR(20),
  target_name  VARCHAR(100)
);
```

Tabla B3

Descripción de campos de la tabla scan_reports

Campo	Tipo de dato	Descripción
id	BIGSERIAL	Clave primaria generada automáticamente.
executed_at	TIMESTAMP	Fecha y hora de ejecución del escaneo.
total_detected	INTEGER	Total de vulnerabilidades detectadas en la corrida.
audited	INTEGER	Cantidad de vulnerabilidades que pasaron al proceso de auditoría.
ignored	INTEGER	Cantidad de vulnerabilidades descartadas (whitelist).

criticals / highs / mediums / lows	INTEGER	Conteo de vulnerabilidades por nivel de severidad.
system_status	VARCHAR(50)	Estado general del sistema al momento del escaneo (ej.: OK, WARNING).
report_message	TEXT	Mensaje descriptivo del resultado del escaneo.
environment_breakdown	JSONB	Desglose de resultados por entorno, almacenado en formato JSON.
target_type / target_name	VARCHAR	Tipo y nombre del objetivo del escaneo (ej.: entorno, activo específico).

Nota. Los tipos de dato corresponden a la nomenclatura de PostgreSQL 14. FK = Foreign Key. Elaboración propia (2026).

B.4. Tabla vulnerabilities_audit

Constituye el registro central de hallazgos del sistema. Cada fila representa una vulnerabilidad detectada y vinculada a un activo específico. Almacena el identificador CVE o GHSA, la puntuación CVSS, el indicador de explotación activa (CISA KEV), la prioridad calculada por el motor de riesgo, la decisión tomada y el estado de gestión.

Script de creación:

```
CREATE TABLE IF NOT EXISTS public.vulnerabilities_audit (
    id          BIGSERIAL PRIMARY KEY,
    detected_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    updated_at  TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    asset_id    BIGINT,
    cve_id      VARCHAR(50),
    ghsa_id     VARCHAR(30),
    cvss        VARCHAR(20),
    exploited   BOOLEAN      DEFAULT FALSE,
    priority    VARCHAR(20),
    decision    TEXT,
    assigned_to VARCHAR(100) DEFAULT 'SIN_ASIGNAR',
    status      VARCHAR(20)  DEFAULT 'DETECTADA',
    resolved_at TIMESTAMPTZ,
    CONSTRAINT fk_audit_asset
    FOREIGN KEY (asset_id)
    REFERENCES public.assets (id) ON DELETE CASCADE
);
```


Tabla B4

Descripción de campos de la tabla vulnerabilities_audit

Campo	Tipo de dato	Descripción
id	BIGSERIAL	Clave primaria generada automáticamente.
detected_at	TIMESTAMP	Fecha y hora en que se detectó la vulnerabilidad.
updated_at	TIMESTAMP	Fecha y hora de la última actualización del registro.
asset_id	BIGINT (FK)	Referencia al activo afectado (FK → assets.id).
cve_id	VARCHAR(50)	Identificador CVE de la vulnerabilidad (ej.: CVE-2024-12345).
ghsa_id	VARCHAR(30)	Identificador GHSA de la GitHub Advisory Database.
cvss	VARCHAR(20)	Puntuación CVSS de severidad técnica.
exploited	BOOLEAN	Indica si la vulnerabilidad está confirmada en el catálogo CISA KEV.
priority	VARCHAR(20)	Prioridad calculada por el motor JS: CRITICAL, HIGH, MEDIUM o LOW.
decision	TEXT	Plan de acción sugerido o decisión tomada sobre la vulnerabilidad.
assigned_to	VARCHAR(100)	Responsable asignado para la gestión. Por defecto: SIN_ASIGNAR.
status	VARCHAR(20)	Estado actual del hallazgo (ej.: DETECTADA, EN_GESTIÓN, RESUELTA).
resolved_at	TIMESTAMP	Fecha y hora de resolución. Nulo mientras el hallazgo esté abierto.

Nota. Los tipos de dato corresponden a la nomenclatura de PostgreSQL 14. FK = Foreign Key. Elaboración propia (2026).

B.5. Tabla vulnerabilities_ignored

Implementa la whitelist de vulnerabilidades descartadas por el sistema. Registra aquellos CVEs que fueron analizados y clasificados como falsos positivos o riesgos aceptados, evitando que generen notificaciones repetidas. El campo `hit_count` se incrementa cada vez que el pipeline vuelve a detectar una entrada ya descartada.

Script de creación:

```
CREATE TABLE IF NOT EXISTS public.vulnerabilities_ignored (
    id          BIGSERIAL PRIMARY KEY,
    package_name VARCHAR(255),
    ecosystem   VARCHAR(50),
    vulnerable_range VARCHAR(100),
    patched_version VARCHAR(100),
    ignored_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    hit_count   INTEGER     DEFAULT 1,
    CONSTRAINT unique_package_vulnerability
    UNIQUE (package_name, vulnerable_range)
);
```

Tabla B5

Descripción de campos de la tabla `vulnerabilities_ignored`

Campo	Tipo de dato	Descripción
<code>id</code>	BIGSERIAL	Clave primaria generada automáticamente.
<code>package_name</code>	VARCHAR(255)	Nombre del paquete al que pertenece la vulnerabilidad descartada.
<code>ecosystem</code>	VARCHAR(50)	Ecosistema del paquete (ej.: npm, pip).
<code>vulnerable_range</code>	VARCHAR(100)	Rango de versiones afectadas por la vulnerabilidad.
<code>patched_version</code>	VARCHAR(100)	Versión en la que se publicó el parche, si existe.
<code>ignored_at</code>	TIMESTAMP	Fecha y hora del último descarte registrado.
<code>hit_count</code>	INTEGER	Cantidad de veces que el pipeline detectó y descartó esta vulnerabilidad.

Nota. Los tipos de dato corresponden a la nomenclatura de PostgreSQL 14. FK = Foreign Key. Elaboración propia (2026).

B.6. Tabla users

Gestiona los usuarios con acceso al sistema de gestión de vulnerabilidades. Permite controlar quién puede consultar y operar sobre los datos registrados.

Script de creación:

```
CREATE TABLE IF NOT EXISTS public.users (
    id      BIGSERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    full_name VARCHAR(100),
    email   VARCHAR(100),
    role    VARCHAR(20) DEFAULT 'AUDITOR',
    created_at TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
);
```

Tabla B6

Descripción de campos de la tabla users

Campo	Tipo de dato	Descripción
id	BIGSERIAL	Clave primaria generada automáticamente.
username	VARCHAR(50)	Nombre de usuario único para autenticación.
full_name	VARCHAR(100)	Nombre completo del usuario.
email	VARCHAR(100)	Dirección de correo electrónico del usuario.
role	VARCHAR(20)	Rol asignado al usuario (ej.: AUDITOR, ADMIN). Por defecto: AUDITOR.
created_at	TIMESTAMP	Fecha y hora de creación del usuario.

Nota. Los tipos de dato corresponden a la nomenclatura de PostgreSQL 14. FK = Foreign Key. Elaboración propia (2026).

B.7. Tabla system_errors

Registra los errores producidos durante la ejecución de los nodos del workflow en n8n, permitiendo la trazabilidad de fallos y su posterior análisis. Cada fila corresponde a un error capturado por el flujo de errores del pipeline.

Script de creación:

```
CREATE TABLE IF NOT EXISTS public.system_errors (
    id      BIGSERIAL PRIMARY KEY,
    error_date  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    node_name  VARCHAR(100),
    error_message TEXT,
    workflow_id VARCHAR(50),
    execution_id VARCHAR(50)
);
```

Tabla B7

Descripción de campos de la tabla system_errors

Campo	Tipo de dato	Descripción
id	BIGSERIAL	Clave primaria generada automáticamente.
error_date	TIMESTAMP	Fecha y hora en que se produjo el error.
node_name	VARCHAR(100)	Nombre del nodo de n8n donde ocurrió el fallo.
error_message	TEXT	Mensaje de error completo para diagnóstico.
workflow_id	VARCHAR(50)	Identificador del workflow en n8n.
execution_id	VARCHAR(50)	Identificador de la ejecución específica donde se produjo el error.

Nota. Los tipos de dato corresponden a la nomenclatura de PostgreSQL 14. FK = Foreign Key. Elaboración propia (2026).

B.8. Relaciones entre Tablas

El esquema implementa dos relaciones de clave foránea que aseguran la integridad referencial entre los datos. Ambas están definidas con la cláusula ON DELETE CASCADE, de modo que la eliminación de un registro padre elimina automáticamente los registros dependientes.

Tabla B8

Relaciones de clave foránea del esquema GEF Secure

Tabla origen	Columna FK	Tabla destino	Columna referenciada	Tipo de relación
assets	environment_id	environments	id	N:1 — Un entorno puede tener muchos activos.
vulnerabilities_audit	asset_id	assets	id	N:1 — Un activo puede tener múltiples registros de auditoría.

Nota. Las relaciones se implementaron con ON DELETE CASCADE para garantizar consistencia referencial. La vista view_vulnerability_details consolida ambas relaciones en una consulta unificada que expone el nombre del activo y su entorno junto al hallazgo. Elaboración propia (2026).

B.9. Vista Auxiliar: view_vulnerability_details

El esquema incluye una vista que consolida información de las tablas vulnerabilities_audit, assets y environments, simplificando las consultas desde el backend y el frontend del sistema. La vista expone el identificador de auditoría, el nombre del activo, el nombre del entorno, el identificador CVE, la prioridad y el estado de cada hallazgo.

Definición de la vista:

```
CREATE OR REPLACE VIEW public.view_vulnerability_details AS
SELECT
    v.id      AS audit_id,
    a.name     AS asset_name,
    e.name     AS environment_name,
    v.cve_id,
    v.priority,
    v.status,
    v.detected_at
FROM public.vulnerabilities_audit v
JOIN public.assets a ON v.asset_id = a.id
JOIN public.environments e ON a.environment_id = e.id;
```

Nota. La vista no almacena datos; constituye una consulta predefinida sobre las tablas base. Su uso desde el backend reduce la complejidad de las consultas y mejora la mantenibilidad del código. Elaboración propia (2026).

Anexo C — Workflow de automatización en n8n

El presente anexo describe la arquitectura del workflow de automatización implementado en la plataforma n8n para el sistema GEF Secure. El pipeline se organiza en tres flujos independientes: el Flujo Principal, responsable del ciclo completo de detección, análisis y notificación de vulnerabilidades; el Flujo Inyector, utilizado durante el desarrollo para poblar el inventario de activos con datos de prueba reales; y el Flujo de Errores, encargado de capturar y registrar fallos de ejecución en cualquier nodo del sistema.

Cada flujo se describe mediante: (a) una explicación funcional de su propósito y lógica de operación, (b) un diagrama de flujo textual que representa la secuencia y ramificaciones de nodos, y (c) una tabla con el detalle de cada nodo, su tipo y su función específica dentro del flujo.

C.1. Flujo Principal

El Flujo Principal constituye el núcleo operativo del sistema. Puede ser iniciado de dos formas: de manera programada mediante el nodo Scan_Scheduler, que lo ejecuta diariamente a las 09:00 horas, o bajo demanda a través del nodo Webhook, que expone el endpoint POST /auditoria-automatizada para recibir solicitudes desde el frontend o sistemas externos. En ambos casos, el nodo Edit Fields normaliza los parámetros de filtro antes de continuar.

Una vez iniciado, el flujo recolecta inteligencia de amenazas desde dos fuentes externas en paralelo: el catálogo CISA KEV, que identifica vulnerabilidades con explotación activa confirmada, y la GitHub Advisory Database, que provee advisories técnicos con información de paquetes afectados. Los datos obtenidos se descomponen, deduplicados y correlacionados con el inventario interno de activos almacenado en PostgreSQL.

Los activos coincidentes avanzan hacia el motor de priorización, implementado en JavaScript, que asigna un nivel de riesgo (CRITICAL, HIGH, MEDIUM o LOW) en función de la presencia en CISA KEV y la puntuación CVSS. Los hallazgos son persistidos en la tabla vulnerabilities_audit, y las vulnerabilidades sin coincidencia de activo son registradas en la whitelist vulnerabilities_ignored. Al finalizar, el flujo calcula métricas agregadas, las persiste en scan_reports y envía un reporte de postura de seguridad a Slack, adaptado al alcance del escaneo (global, por entorno o por activo). Las vulnerabilidades de prioridad CRITICAL y HIGH generan además una alerta individual inmediata.

Figura C1

Diagrama del Flujo Principal del sistema GEF Secure

(IMAGEN FLUJO PRINCIPAL)

Nota. El diagrama representa la secuencia lógica de nodos. Las ramas indican puntos de decisión (nodos Switch). Los nodos de persistencia en PostgreSQL y notificación en Slack operan en paralelo con la rama principal en varios puntos del flujo. Elaboración propia (2026).

Tabla C1

Nodos del Flujo Principal y su función

Nodo	Tipo	Función en el flujo
Scan_Scheduler	Schedule Trigger	Dispara el flujo principal diariamente a las 09:00 hs.
Webhook	Webhook (POST)	Recibe solicitudes de auditoría desde el frontend o sistemas externos vía HTTP POST a /auditoria-automatizada.
Edit Fields	Set	Normaliza los parámetros de filtro (filtro_entorno, filtro_activo) recibidos del webhook o los inicializa en 'TODOS' cuando el disparador es el scheduler.
Fetch_CISA_KEV	HTTP Request	Descarga el catálogo de vulnerabilidades activamente explotadas desde la API pública de CISA (known_exploited_vulnerabilities.json).
Fetch_GitHub_Advisories	HTTP Request	Consulta la API de GitHub Security Advisories filtrando avisos revisados y de severidad crítica.
Extract_Advisory_Items	Split Out	Descompone el array de vulnerabilidades de cada advisory en ítems individuales para procesamiento unitario.
Deduplicate_Advisories	Remove Duplicates	Elimina entradas repetidas comparando por ghsa_id, evitando el reprocesamiento de advisories ya vistos en la misma corrida.

Check_Internal_Assets	PostgreSQL	Ejecuta una consulta SQL que busca coincidencias entre el software del advisory y el inventario interno de activos, respetando los filtros de entorno y activo.
Filter_Matched_Assets	Switch	Separa el flujo según si se encontraron activos coincidentes (asset_name no vacío) o no. Los no coincidentes se derivan a descarte.
Deduplicate_Target_Assets	Remove Duplicates	Elimina activos duplicados comparando por software, ecosystem y env_name antes de la fase de enriquecimiento.
Enrich_Vulnerability_Data	Merge	Combina la inteligencia del advisory con los datos del activo interno, generando un objeto unificado con riesgo técnico y contexto operativo.
Validate_Asset_Match	Switch	Verifica que el campo software no esté vacío. Si lo está, el ítem se deriva a la whitelist; si tiene valor, avanza al motor de riesgo.
Log_Ignored_Vulnerabilities	PostgreSQL	Inserta o actualiza en vulnerabilities_ignored las entradas descartadas. Implementa lógica UPSERT: si el CVE ya existe, incrementa hit_count.
Risk_Assessment_Engine	Code (JavaScript)	Motor de priorización: asigna CRITICAL si el CVE está en CISA KEV, HIGH si CVSS $\geq$ 9.0, MEDIUM u LOW según severidad residual. Genera el plan de acción.
Persist_Audit_Findings	PostgreSQL	Persiste cada hallazgo auditado en la tabla vulnerabilities_audit con todos sus metadatos.
Remove Duplicates	Remove Duplicates	Deduplica por asset_id antes de actualizar el timestamp de escaneo.
Update_Asset_Audit_Timestamp	PostgreSQL	Actualiza el campo last_scan del activo en la tabla assets.

Consolidate_Audit_Results	Merge	Reúne los resultados de la ruta de ignorados y la ruta de auditados para el cálculo de métricas globales.
Compute_Audit_Metrics	Code (JavaScript)	Calcula métricas agregadas: total procesado, auditados, ignorados, conteo por severidad y desglose por entorno.
Persist_Scan_Statistics	PostgreSQL	Inserta el resumen estadístico de la ejecución en la tabla scan_reports.
Switch1	Switch	Determina el alcance del reporte de estado (GLOBAL, ENTORNO o ACTIVO) según los filtros activos.
Notify_Security_Status	Slack	Envía reporte global de postura de seguridad al canal de Slack con desglose por entorno.
Notify_Security_Status1_entorno	Slack	Envía reporte segmentado por entorno cuando el filtro_entorno está activo.
Notify_Security_Status2_activo	Slack	Envía reporte técnico de activo específico cuando el filtro_activo está activo.
Switch	Switch	Enruta cada hallazgo según su prioridad: CRITICAL y HIGH disparan alerta inmediata en Slack; MEDIUM y LOW solo se registran.
Send a message	Slack	Envía la alerta individual de vulnerabilidad al canal de seguridad con CVE, CVSS, activo afectado y plan de acción.

Nota. El tipo 'Code (JavaScript)' corresponde al nodo nativo de n8n que ejecuta scripts JavaScript en el servidor. Los nodos PostgreSQL utilizan la versión 2.6 del conector oficial de n8n. Elaboración propia (2026).

C.2. Flujo Inyector

El Flujo Inyector es un flujo auxiliar diseñado para poblar el inventario de activos con datos de prueba extraídos de advisories reales. Su propósito es facilitar el desarrollo y la validación del sistema en entornos de laboratorio, asegurando que el Flujo Principal cuente con activos registrados sobre los cuales pueda detectar vulnerabilidades.

Al ser disparado manualmente, el flujo consulta GitHub Advisories filtrando por severidad crítica, selecciona aleatoriamente uno de los paquetes vulnerables obtenidos, extrae su versión y lo asigna a uno de los tres entornos disponibles (Desarrollo, Testing o Producción) de forma aleatoria. El activo es insertado o actualizado en la tabla assets mediante una operación UPSERT, y se envía una confirmación al canal de Slack del equipo de desarrollo.

Figura C2

Diagrama del Flujo Inyector del sistema GEF Secure

(IMAGEN DE FLUJO INYECTOR)

Nota. Este flujo opera de forma completamente independiente del Flujo Principal. Su ejecución no afecta el estado de vulnerabilities_audit ni scan_reports. Elaboración propia (2026).

Tabla C2

Nodos del Flujo Inyector y su función

Nodo	Tipo	Función en el flujo
Manual_Trigger	Manual Trigger	Disparador manual que inicia el flujo inyector desde la interfaz de n8n.

Fetch_Global_Advisories	HTTP Request	Consulta GitHub Advisories con filtro de severidad crítica para obtener el pool de vulnerabilidades reales a inyectar.
Random_Asset_Selector	Code (JavaScript)	Selecciona aleatoriamente un paquete vulnerable del pool, extrae su versión y lo asigna a un entorno aleatorio (ID 1, 2 o 3).
Upsert_Test_Asset	PostgreSQL	Inserta o actualiza el activo de prueba en la tabla assets con ON CONFLICT, evitando duplicados.
Notify_Dev_Slack1	Slack	Confirma al equipo de desarrollo que el activo fue inyectado correctamente, indicando software, versión, ecosistema y entorno asignado.

Nota. Elaboración propia (2026).

C.3. Flujo de Errores

El Flujo de Errores implementa un mecanismo centralizado de captura y trazabilidad de fallos. El nodo Error_Trigger es un disparador nativo de n8n que se activa automáticamente cuando cualquier nodo del workflow lanza una excepción no controlada, independientemente del flujo en el que ocurra.

Al activarse, el flujo ejecuta dos acciones en paralelo: registra el error en la tabla system_errors de PostgreSQL, incluyendo el nombre del nodo fallido, el mensaje de error, el identificador del workflow y el identificador de la ejecución; y envía una notificación inmediata al canal de Slack del equipo de desarrollo con la misma información, permitiendo una respuesta rápida ante incidentes de infraestructura.

Figura C3

Diagrama del Flujo de Errores del sistema GEF Secure

(IMAGEN FLUJO DE ERRORES)

Nota. El nodo Error_Trigger de n8n captura errores de todos los flujos del workflow, no solo del Flujo Principal. Esto garantiza cobertura de trazabilidad para cualquier falla del sistema. Elaboración propia (2026).

Tabla C3

Nodos del Flujo de Errores y su función

Nodo	Tipo	Función en el flujo
Error_Trigger	Error Trigger	Se activa automáticamente cuando cualquier nodo del workflow lanza un error no controlado.
Log_System_Failure	PostgreSQL	Registra el error en la tabla system_errors con el nombre del nodo fallido, el mensaje de error, el ID del workflow y el ID de ejecución.
Notify_Dev_Slack	Slack	Notifica al canal de desarrollo el fallo de infraestructura con el nodo afectado y el mensaje de error para diagnóstico inmediato.

Nota. Elaboración propia (2026).

Anexo D — Ejemplo de notificación en Slack

El presente anexo ilustra los tres tipos de notificaciones automáticas generadas por el sistema GEF Secure hacia el canal de Slack del equipo de seguridad. Cada tipo de notificación corresponde a un momento distinto del flujo de procesamiento: alertas individuales por vulnerabilidad crítica o alta, reportes de postura de seguridad al finalizar un escaneo, y confirmaciones del flujo inyector durante el desarrollo.

D.1. Alerta Individual de Vulnerabilidad (CRITICAL / HIGH)

Este tipo de notificación se genera por cada vulnerabilidad cuya prioridad calculada sea CRITICAL o HIGH. Es disparada por el nodo Send a message inmediatamente después de que el motor de riesgo asigna la prioridad, sin esperar a que finalice el ciclo completo de escaneo.

Figura D1

Ejemplo de alerta individual de vulnerabilidad crítica enviada al canal de Slack

 **INSERTAR CAPTURA:** Ejecutá el workflow con un activo que tenga una vulnerabilidad CRITICAL o HIGH y capturá el mensaje recibido en Slack. Debe mostrar el bloque completo con CVE, GHSA, CVSS, activo afectado y plan de acción.

Nota. La notificación incluye el identificador CVE, el identificador GHSA, la puntuación CVSS, el nombre del activo afectado, la indicación de si la vulnerabilidad figura en el catálogo CISA KEV y el plan de acción sugerido por el motor de priorización. Las prioridades MEDIUM y LOW no generan este tipo de alerta; únicamente quedan registradas en la base de datos. Elaboración propia (2026).

D.2. Reporte de Postura de Seguridad (Cierre de Escaneo)

Al finalizar cada ejecución del Flujo Principal, el nodo Switch1 determina el alcance del reporte (global, por entorno o por activo) y despacha la notificación correspondiente. El reporte global es enviado por Notify_Security_Status e incluye el desglose completo de métricas de la corrida.

Figura D2

Ejemplo de reporte global de postura de seguridad al finalizar un escaneo

 **INSERTAR CAPTURA:** Ejecutá el workflow completo con filtro_entorno = "TODO" y capturá el mensaje de cierre en Slack. Debe mostrar el ID de ejecución, fecha, totales por severidad, desglose por entorno y la postura de seguridad final (ESTABLE o ACCIÓN REQUERIDA).

Nota. Cuando el filtro activo corresponde a un entorno específico, la notificación es enviada por el nodo Notify_Security_Status1_entorno con métricas segmentadas. Cuando corresponde a un activo puntual, la notificación es enviada por Notify_Security_Status2_activo con el resultado de auditoría de ese activo. Elaboración propia (2026).

D.3. Confirmación del Flujo Inyector

Durante la fase de desarrollo y validación, el Flujo Inyector envía una confirmación al canal de Slack cada vez que inserta un activo de prueba en la base de datos, permitiendo al equipo verificar que la inyección se realizó correctamente antes de ejecutar el Flujo Principal.

Figura D3

Ejemplo de notificación de confirmación del Flujo Inyector

 **INSERTAR CAPTURA:** Ejecutá el Flujo Inyector manualmente desde n8n y capturá el mensaje de confirmación en Slack. Debe mostrar el nombre del activo, software, ecosistema, versión, severidad reportada y el texto de confirmación de persistencia.

Nota. Esta notificación es generada exclusivamente por el nodo Notify_Dev_Slack1 del Flujo Inyector y no forma parte del ciclo operativo normal del sistema. Su propósito es exclusivamente de validación durante el desarrollo. Elaboración propia (2026).

Anexo E — Endpoints del backend

Este anexo describe los principales endpoints implementados en la API del sistema, desarrollada en Java.

(Acá listamos los endpoints reales) - Sale de: *tu backend (Spring Boot)*

Ejemplo:

- GET /vulnerabilities
- GET /vulnerabilities/{id}
- PUT /vulnerabilities/{id}/status

Anexo F — Ejemplo de datos procesados

El presente anexo ilustra la estructura de los datos generados y almacenados por el sistema GEF Secure durante un ciclo completo de escaneo. Se presentan tres ejemplos representativos: la salida del motor de priorización en formato JSON, un registro persistido en la tabla de auditoría y el resumen estadístico de una ejecución almacenado en la tabla de reportes.

F.1. Salida del Motor de Priorización (Risk_Assessment_Engine)

El nodo Risk_Assessment_Engine produce un objeto JSON por cada vulnerabilidad procesada. El siguiente ejemplo corresponde a una vulnerabilidad clasificada como CRITICAL por estar confirmada en el catálogo CISA KEV.

Tabla F1

Ejemplo de objeto JSON generado por el motor de priorización ante una vulnerabilidad crítica

```
{
  "asset_name": "ASSET-EXPRESS",
  "software": "express",
  "ecosystem": "npm",
  "version_instalada": "4.17.1",
  "cve_id": "CVE-2024-29041",
  "ghsa_id": "GHSA-rv95-896h-c2vc",
  "cvss": "7.5",
  "is_cisa_kev": true,
  "priority": "CRITICAL",
  "action_plan": "Vulnerabilidad con explotación activa confirmada (CISA KEV). Actualizar express a la versión parcheada de forma inmediata. Revisar dependencias transitivas del ecosistema npm.",
  "environment": "Producción",
  "env_criticality": "ALTA",
  "processed_at": "21/04/2026, 09:03:47"
}
```

Nota. El campo is_cisa_kev indica si la vulnerabilidad figura en el catálogo Known Exploited Vulnerabilities de CISA. Cuando este campo es true, el motor asigna prioridad CRITICAL independientemente del valor CVSS. El campo action_plan es generado dinámicamente por el script JavaScript del nodo. Elaboración propia (2026).

F.2. Registro en la Tabla vulnerabilities_audit

Una vez procesado por el motor de priorización, cada hallazgo es persistido en la tabla vulnerabilities_audit. El siguiente ejemplo muestra la estructura de un registro real tal como quedaría almacenado en PostgreSQL.

Tabla F2

Ejemplo de registro persistido en la tabla vulnerabilities_audit

Campo	Valor Ejemplo
id	142
detected_at	2026-04-21 09:03:47
updated_at	2026-04-21 09:03:47
asset_id	7
cve_id	CVE-2024-29041
ghsa_id	GHSA-rv95-896h-c2vc
cvss	7.5
exploited	true
priority	CRITICAL
decision	Vulnerabilidad con explotación activa confirmada (CISA KEV). Actualizar express a la version parcheada de forma inmediata.
assigned_to	SIN_ASIGNAR
status	DETECTADA
resolved_at	NULL

Nota. El campo resolved_at permanece NULL mientras el hallazgo no haya sido gestionado. El campo assigned_to es actualizado manualmente desde el frontend o mediante la API REST cuando un responsable toma a cargo la vulnerabilidad. Elaboración propia (2026).

F.3. Registro en la Tabla scan_reports

Al finalizar cada ejecución del Flujo Principal, el nodo Persist_Scan_Statistics almacena un resumen estadístico de la corrida en la tabla scan_reports. El siguiente ejemplo corresponde a un escaneo global con hallazgos de distintas severidades.

Tabla F3

Ejemplo de registro persistido en la tabla scan_reports tras un escaneo global

Campo	Valor Ejemplo
id	38
executed_at	2026-04-21 09:03:51
total_detected	24
audited	18
ignored	6
criticals	2
highs	5
mediums	8
lows	3
system_status	⚠ ACCIÓN REQUERIDA
report_message	Análisis finalizado_ 24 registros evaluados.
environment_breakdown	{"Producción": {"criticos": 2, "altas": 3, "medias": 4, "bajas": 1, "total": 10}, "Testing": {"criticos": 0, "altas": 2, "medias": 4, "bajas": 2, "total": 8}}
target_type	GLOBAL
target_name	TODOS

Nota. El campo environment_breakdown almacena el desglose de hallazgos por entorno en formato JSONB, lo que permite consultas analíticas directamente desde PostgreSQL. El valor ⚠ ACCIÓN REQUERIDA en system_status es asignado automáticamente cuando criticals > 0. Elaboración propia (2026).

Anexo G — Interfaz de usuario (Frontend)

Este anexo muestra la interfaz desarrollada para la visualización de la información, incluyendo ejemplos de consultas y visualización de vulnerabilidades.

(Acá ponemos capturas del frontend) - Sale de: *tu implementación de frontend*

TENER EN CUENTA: Los anexos tienen que tener contexto, osea tenemos que relacionarlos con la parte del informe a la que estén haciendo referencia por lo que hay que agregar estas modificaciones en donde se mencione cada anexo.

Por ejemplo: (dentro del capítulo 4 o 5) agregar algo como: "El workflow implementado puede observarse en el Anexo C". y así con todos.